

• AI DELIVERY MANAGER · ENTERPRISE SOLUTIONS

Delivering Enterprise AI That Actually Ships

Bridging the gap between **AI engineering teams** and **enterprise product outcomes** — coordinating multi-disciplinary AI pods from POC to production at scale.

Enterprise AI Delivery · 2025–2026

87.5%

TIME SAVED · TVL CONVERTER

75%

FASTER VARIANCE RESOLUTION

85%

TIME SAVED · EDI GENERATION

3 AI

PRODUCTION POCS DELIVERED

 View AI Projects

 Tech Stack

ABOUT

AI Delivery Manager & Delivery Lead

Working at the intersection of enterprise product migration and AI engineering, I coordinate end-to-end delivery of AI-powered solutions — from scoping and stakeholder alignment to deployment and adoption. My projects span agentic systems, LLM integration, and automation pipelines that dramatically compress engineering effort.



Stakeholder Alignment

Bridging client engineering teams and AI delivery pods, driving consensus on scope, access, and success criteria.



AI Pipeline Coordination

Overseeing end-to-end AI pipelines: LLM integration, agentic workflows via GitHub Copilot, Jira MCP, and Claude APIs.



Metrics & Impact Tracking

Quantifying engineering effort reduction, accuracy improvements, and scalability gains across enterprise migration programs.



POC-to-Production

Driving proof-of-concept AI solutions through validation, stakeholder review, and phased production rollout.

AI PROJECTS

Three Delivered AI Solutions

All three projects were delivered for an **Enterprise HCM Platform Client**, targeting real engineering bottlenecks in enterprise-scale migration and benefits processing programs.



Project 1 · AI-Powered EDI File Generation

Benefits Platform

Claude API

GitHub Copilot

XML / EDI 834

Jira MCP

April 2026

★ PROBLEM

Benefits engineers manually built EDI 834 XML files for insurance carriers — a slow,

🏗️ AI SOLUTION ARCHITECTURE

- **Jira** Connects to Jira, pulls carrier specs & group IDs from tickets
- **Agent:**

error-prone process requiring deep domain expertise. With 20,000+ existing XML files and peak volumes of 10–20 new builds per day, the process was a critical bottleneck.

**20 min–
2 hr**

PER EDI BUILD
(BEFORE)

~15 min

PER EDI BUILD
(AFTER)

85%

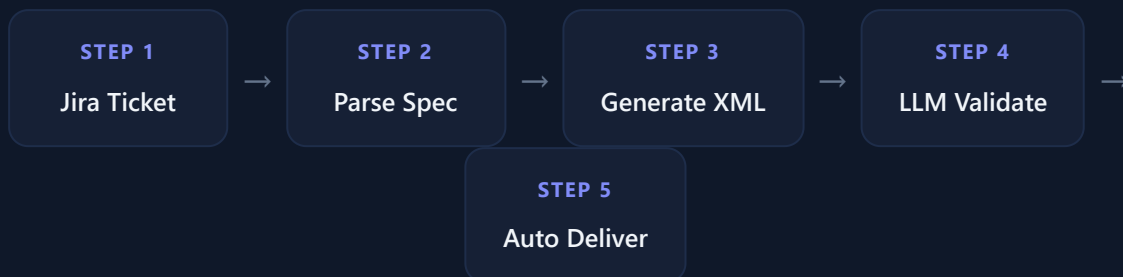
TIME SAVINGS

20K+

XML FILES IN
LIBRARY

- ▶ **Spec Parser:** LLM reads Excel/PDF/Word specs, extracts field mappings & usage flags
- ▶ **XML Generator:** Claude API produces DSL-based XML using 20K+ file library as reference
- ▶ **Auto Validator:** Iterative LLM loop checks for missing segments, incorrect IDs, format errors
- ▶ **Auto Delivery:** Generated file is attached to carrier contract, exported & verified automatically
- ▶ **GitHub Copilot UI:** Engineers trigger and adjust pipeline through the enterprise-approved Copilot interface

AI PIPELINE FLOW



BEFORE AI

Manual copy-paste XML from similar files

20 min – 2 hrs per new carrier build

Carrier feedback loops causing rework

Peer code review required every time

Peak load: 15–20 builds/day unsustainable

AFTER AI

AI reads spec → auto-generates XML template

~15 min per build, consistent quality

Built-in LLM validation catches errors first

Auto validation replaces manual code review

Scalable to any peak volume without extra FTEs



Project 2 · AI TVL Converter — Legacy-to-Modern Migration

Migration Program

Pay Compare UI

LLM API

CC → EC Migration

 PROBLEM

The client had 1,000+ customers on a legacy Classic Calc (CC) engine (Delphi, 1990s) needing migration to Enhanced Calc (EC, .NET). Engineers manually translated complex TVL (embedded SQL-like) logic — 4–6 hours per expression — making a 3+ year migration timeline unsustainable.

4–6 hrsPER EXPRESSION
(BEFORE)**30 min**PER EXPRESSION
(AFTER)**87.5%**

TIME SAVINGS

80%ACCURACY
ACHIEVED AI SOLUTION CAPABILITIES

- ▶ **Automated Conversion:** AI-powered Customs Converter integrated directly into Pay Compare UI
- ▶ **EC-compliant Output:** Returns validated JSON output conforming to Enhanced Calc schema
- ▶ **Domain-Aware LLM:** Handles domain-specific rules, patterns, and edge cases automatically
- ▶ **Dual API Endpoints:** One for conversion, one for business-friendly step-by-step explanation
- ▶ **Seamless Integration:** Fully accessible through REST API integrated with Pay Compare
- ▶ **Continuous Refinement:** Feedback loop for ongoing prompt tuning and model improvement

 BEFORE AI

Manual TVL → EC translation by domain experts

4–6 hours per expression, a serious bottleneck

Human errors, logic inconsistencies, heavy QA

High dependency on rare domain experts

Cannot scale — 3+ year migration timeline

 AFTER AI

Instant AI conversion via API or Pay Compare UI

30 minutes — engineer validates only

Standardized, uniform EC expressions

AI handles domain rules — experts freed up

Scalable to high volume — timeline compressed



Project 3 · AI Variance Analysis — Root Cause Automation

Legacy Migration

Claude Sonnet 4.5

GitHub Copilot Agents

Jira MCP

ECE Log Analysis

Dec 2025

Calculation variances between Classic Calc and Enhanced Calc were reviewed manually — a slow, expert-dependent process that delayed customer migrations. Engineers dug through ECE logs, Jira tickets, and attachments, taking 6–12 hours per variance with inconsistent results.

6–12 hrs

PER VARIANCE
(BEFORE)

≤3 hrs

PER VARIANCE
(TARGET)

75%

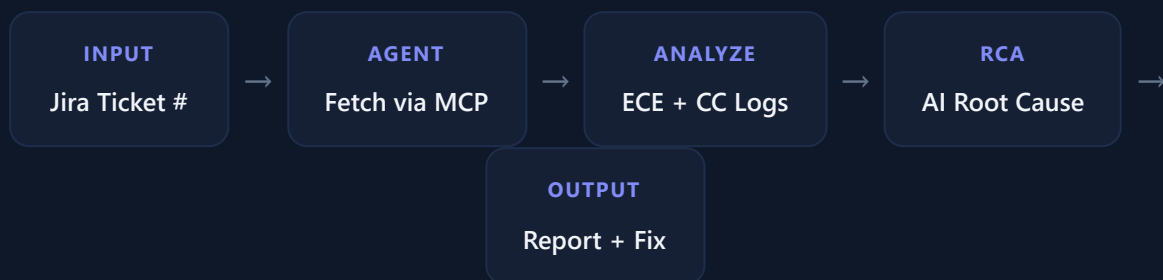
TIME REDUCTION

90%

TARGET ACCURACY

- ▶ **Orchestrator Agent:** GitHub Copilot Agent takes Jira ticket # and drives the full workflow
- ▶ **Jira MCP Tools:** Fetches ticket info, downloads attachments, surfaces related historical tickets
- ▶ **ECE Log Analyzer:** Calc log analysis and backtracking through Claude Sonnet 4.5
- ▶ **CCE Analyzer:** Identifies code pointers, function calls, and branching logic mismatches
- ▶ **Excel Parser:** Processes variance analysis spreadsheets automatically
- ▶ **Consolidated RCA Report:** AI generates clear, business-friendly root cause explanation

AGENTIC WORKFLOW



BEFORE AI

Manual ECE log review — highly expertise-dependent

6–12 hours per variance ticket

Human errors, inconsistent RCA categorization

Manual Jira ticket trawling for historical patterns

Cannot scale with growing migration volume

AFTER AI

Claude Sonnet 4.5 automates extraction & comparison

Target: ≤3 hours per variance resolution

AI-driven parsing — standardized variance categories

Jira MCP surfaces related tickets automatically

Scales across all customers in parallel

TECHNOLOGY

Tech Stack & Tools

Hands-on experience coordinating and delivering solutions built with modern AI infrastructure, agentic frameworks, and enterprise integrations.



Claude API

LLM · Anthropic



GitHub Copilot

AI IDE · Agents



Jira MCP

Model Context Protocol



Python

Core Language



Jira

Project Tracking



REST API

Integration Layer



Gradio

AI UI / Web App



EDI 834 XML

Benefits Data Format



Pay Compare

Enterprise Platform



Everything Benefits

Benefits Platform



DSL Engine

Custom Transformation



GitHub

Version Control

JOURNEY

Project Timeline

Chronological delivery of AI solutions across the enterprise legacy migration program and benefits automation platform.

DECEMBER 2025

AI TVL Converter — Production POC Launch

Delivered the first AI solution converting TVL expressions from Classic Calc to Enhanced Calc. Achieved 87.5% time savings (4–6 hrs → 30 min per expression), integrated into Pay Compare UI and accessible via REST API. Accuracy: 80%.

DECEMBER 2025

AI Variance Analysis — Agentic System Launch

Built and coordinated delivery of a GitHub Copilot orchestrator agent using Claude Sonnet 4.5 and Jira MCP. Automated ECE log analysis, CC vs EC comparison, RCA generation, and historical ticket intelligence. Target: 75% time reduction and 90% accuracy.

JANUARY 2026

Business Unit Presentation — Migration Program Review

Presented both AI solutions to Business Unit leadership, highlighting the combined impact on migration timelines, engineer productivity, and scalability across 1,000+ customer migrations.

APRIL 2026

AI EDI Generator — Kickoff & POC Architecture

Led kickoff for AI-powered EDI 834 XML generation for the Benefits platform. Coordinated AI engineer team and presented the 6-step pipeline: Jira → Spec Parse → XML Generate → LLM Validate → Attach → Deliver. Projected 85% time savings.

Vivekanandan Nachimuthu

AI Delivery Manager · Enterprise AI Delivery · 2025–2026

 nvivek2822@gmail.com  GitHub

